Лабораторна робота № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали

Постановка

Ви — аналітик у туристичному застосунку. Завдання: для заданого міста **відібрати десять найвартісніших історико-культурних об'єктів** і **спланувати піший маршрут**, який починається на залізничному вокзалі, обходить усі десять і повертається на вокзал.

Слово «найвартісніші» в даних не записане. Критерій ви побудуєте самі — і захистите його.

Потім те саме для **тридцяти** об'єктів. Результат цієї, більшої задачі стане базою порівняння в **лабораторній роботі № 8**, де ту саму задачу розв'язуватимуть генетичним та мурашиним алгоритмами.

Що здаєте

1. Цей ноутбук із виконаними комірками (усі виводи збережені).
2. Файл `lr5_результати.json` — результати для ЛР8.
3. Карту `lr5_маршрут.html` .
4. Звіт у кінці ноутбука: відповіді на сім питань.

Що оцінюється

критерій	0	2	3	4
Відбір об'єктів	теги взято навмання	фільтр працює	фільтр обґрунтовано	показано, що відкинуто і чому
Критерій рейтингу	немає	є формула	ваги пояснено	порівняно з альтернативним критерієм
Маршрут	не будується	будується	перевірено коректність	пояснено різницю методів числом
Карта	немає	точки на карті	маршрут по ребрах графа	видно різницю з прямими відрізками
Звіт	немає	формально	з числами	з висновком, якого не було в завданні

---

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Місто підтягнеться само.

In [39]:
```
STUDENT_NAME = 'Колбасюк Данило'     # напр. 'Коваленко Іван Петрович'
LIST_NUMBER  = 5      # номер у списку групи

CITIES = [
    'Львів', 'Київ', 'Харків', 'Одеса', 'Дніпро',
    'Чернівці', 'Івано-Франківськ', 'Тернопіль', 'Ужгород', 'Луцьк',
    'Рівне', 'Хмельницький', 'Вінниця', 'Житомир', 'Чернігів',
    'Черкаси', 'Полтава', 'Суми', 'Кропивницький', 'Запоріжжя',
    'Миколаїв', 'Кривий Ріг', 'Кам\u2019янець-Подільський', 'Мукачево',
    'Коломия', 'Дрогобич', 'Біла Церква', 'Ніжин', 'Остріг', 'Умань',
]

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у спи
CITY = CITIES[(LIST_NUMBER - 1) % len(CITIES)] + ', Україна'
print('варіант %d: %s' % (LIST_NUMBER, CITY))
```

```
варіант 5: Дніпро, Україна
```

> Якщо викладач призначив вам інше місто — просто перепишіть `CITY` вручну і зазначте це у звіті.

In [40]:
```
import importlib.util, subprocess, sys
REQUIRED = ['osmnx', 'geopandas', 'shapely', 'pyproj', 'pyarrow',
            'folium', 'mapclassify', 'h3', 'networkx']
missing = [p for p in REQUIRED if importlib.util.find_spec(p) is None]
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q',
print('усі пакети на місці')
```

```
усі пакети на місці
```

In [41]:
```
import warnings; warnings.filterwarnings('ignore')
import json, time, os, collections
import numpy as np
import pandas as pd
import geopandas as gpd
import networkx as nx
import matplotlib.pyplot as plt
import folium
from xyzservices import providers as xyz

BASEMAP = xyz.Esri.WorldGrayCanvas   # CARTO з 2026 р. вимагає ключ API
pd.set_option('display.width', 200)
print('geopandas', gpd.__version__)
```

```
geopandas 1.2.0
```

---

1. Два способи дістати дані

Робоча програма вимагає порівняти два способи доступу **за обсягом переданих даних і за часом**:

- **живий запит** — `osmnx` питає Overpass API просто зараз;
- **знімок** — ті самі дані, збережені у файли GeoParquet, читаються з диска.

Нижче — живий запит. Запустіть його **один раз**: він збереже теку зі знімком, і всі наступні запуски читатимуть з неї.

> Overpass — безкоштовний сервіс і обмежує частоту. Якщо запит не пройшов — зачекайте хвилину й повторіть. Якщо не проходить взагалі, візьміть резервний знімок у викладача і вкажіть це у звіті.

In [42]:
```
import osmnx as ox

OUT = 'osm_' + CITY.split(',')[0].strip().lower().replace(' ', '_')
os.makedirs(OUT, exist_ok=True)
ox.settings.use_cache = True
ox.settings.cache_folder = os.path.join(OUT, 'cache')
ox.settings.requests_timeout = 300
ox.settings.overpass_rate_limit = True

TOURISM_OK = ['museum', 'attraction', 'gallery', 'artwork', 'viewpoint']

def cache_bytes():
    import pathlib
    p = pathlib.Path(ox.settings.cache_folder)
    return sum(f.stat().st_size for f in p.glob('*.json')) if p.exists()

print('тека знімка:', OUT)
```

```
тека знімка: osm_дніпро
```

Дві налаштовані дрібниці, без яких запит не пройде

Це не формальності — кожна з них ламає збір по-своєму, і жодна не схожа на свою справжню причину.

Представитись. Overpass відповідає **HTTP 406** клієнтові, який прийшов із типовим заголовком `python-requests`. `osmnx` надсилає свій `User-Agent` сам, але якщо ви звертаєтесь до сервісу напряму — надсилайте теж. Рядок має бути **лише латиницею**: HTTP-заголовки передаються в latin-1, і кирилиця кладе запит помилкою `UnicodeEncodeError` ще до відправлення.

Обрати машину. За іменем `overpass-api.de` стоїть кілька серверів, і частина з них відмовляє у з'єднанні. `osmnx` закріплює одну адресу на всю сесію — навмисно, щоб не порушувати облік слотів. Якщо закріпиться та, що відмовляє, впаде весь збір. Тому адресу обираємо самі, вже перевірену.

In [43]:
```
import socket

USER_AGENT = ('LvivPolytechnic-SK26-coursework/1.0 '
              '(https://lpnu.ua; student lab assignment)')
```

```
assert USER_AGENT.isascii(), 'HTTP-заголовки — latin-1, кирилиця тут не п
ox.settings.http_user_agent = USER_AGENT


def pin_working_ip(hostname, port=443, timeout=8):
    """Закріпити для цього імені адресу, яка справді відповідає."""
    try:
        infos = socket.getaddrinfo(hostname, port, socket.AF_INET, socket
    except Exception as e:
        print('не вдалось резолвити %s: %s' % (hostname, type(e).__name__
        return None
    ips = sorted({i[4][0] for i in infos})
    good = []
    for ip in ips:
        s = socket.socket(); s.settimeout(timeout)
        try:
            s.connect((ip, port)); good.append(ip)
        except Exception:
            pass
        finally:
            s.close()
    print('%s: машин %d, відповідають %d %s' % (hostname, len(ips), len(g
    if not good:
        return None
    original = socket.gethostbyname
    socket.gethostbyname = lambda h: good[0] if h == hostname else origin
    print('закріплено', good[0])
    return good[0]


pin_working_ip('overpass-api.de')
```

```
overpass-api.de: машин 2, відповідають 2 ['162.55.144.139', '65.109.112.5
2']
закріплено 162.55.144.139
```

Out[43]: `'162.55.144.139'`

> Якщо жодна машина не відповіла — виконайте комірку ще раз за хвилину, або візьміть у викладача резервний знімок і зазначте це у звіті. Інші дзеркала перелічені на сторінці Overpass API; адресу можна підставити у `ox.settings.overpass_url`.

Завдання 1.1

Допишіть функцію `fetch`: вона має зберегти шар у `.parquet`, а якщо файл уже є — прочитати його, **не звертаючись до мережі**. Поверніть також, скільки МБ пішло мережею і скільки секунд це зайняло.

Підказка щодо `clean`: у OSM одна колонка може містити і число, і список (ребро, склеєне з кількох шляхів). Parquet вимагає один тип на колонку, тому неоднорідну колонку треба звести до тексту **цілком**.

In [44]:
```
def clean(gdf):
    g = gdf.reset_index()
    for c in g.columns:
```

```
        if c == 'geometry':
            continue
        boxed = g[c].map(lambda v: isinstance(v, (list, dict, set, tuple)
        if boxed or g[c].dtype == object:
            g[c] = g[c].map(
                lambda v: None if v is None or (isinstance(v, float) and
                else (json.dumps(v, ensure_ascii=False)
                      if isinstance(v, (list, dict, set, tuple)) else str
            ).astype('string')
    return g


COST = {}

def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                      'секунд': round(time.time() - t0, 3)}
        return g
    # ----- ВАШ КОД: живий запит -----
    # 1) запам'ятайте cache_bytes() і час ДО виклику getter()
    # 2) викличте getter(), проженіть результат через clean(), збережіть
    # 3) запишіть у COST[name] спосіб 'живий запит', записів, передано_МБ
    #    файл_МБ і секунд
    before=cache_bytes()
    t0=time.time()
    gdf=getter()
    g=clean(gdf)
    g.to_parquet(path)
    after=cache_bytes()
    dt=time.time()-t0
    COST[name]={'спосіб': 'живий запит',
                'записів': len(g),
                'передано_МБ': round((after-before)/1e6, 3),
                'файл_МБ': round(os.path.getsize(path)/1e6, 3),
                'секунд': round(dt, 3)}
    return g
```

In [45]:
```
boundary = fetch('boundary', lambda: ox.geocode_to_gdf(CITY))
poly = boundary.geometry.iloc[0]
print('межа міста отримана, площа %.1f км²'
      % (boundary.to_crs(32635).area.iloc[0] / 1e6))
```

```
межа міста отримана, площа 413.5 км²
```

In [46]:
```
heritage = fetch('heritage', lambda: ox.features_from_polygon(
    poly, {'historic': True, 'tourism': TOURISM_OK}))
stations = fetch('stations', lambda: ox.features_from_polygon(
    poly, {'railway': ['station', 'halt'], 'building': ['train_station']}
amenities = fetch('amenities', lambda: ox.features_from_polygon(
    poly, {'amenity': ['cafe', 'restaurant', 'drinking_water', 'toilets']

# Мережа приходить двома таблицями одразу, тому їй окрема гілка.
NODES_P = os.path.join(OUT, 'road_nodes.parquet')
```

```
EDGES_P = os.path.join(OUT, 'road_edges.parquet')

if os.path.exists(NODES_P) and os.path.exists(EDGES_P):
    t0 = time.time()
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'знімок', 'записів': len(nodes) + le
                           'передано_МБ': 0.0,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P))
                           'секунд': round(time.time() - t0, 3)}
else:
    t0, b0 = time.time(), cache_bytes()
    G_raw = ox.graph_from_polygon(poly, network_type='walk')
    nodes, edges = ox.convert.graph_to_gdfs(G_raw)
    clean(nodes).to_parquet(NODES_P, index=False)
    clean(edges).to_parquet(EDGES_P, index=False)
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'живий запит', 'записів': len(nodes)
                           'передано_МБ': round((cache_bytes() - b0) / 1e
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P))
                           'секунд': round(time.time() - t0, 1)}

print('вузлів %d | ребер %d' % (len(nodes), len(edges)))
```

```
вузлів 50510 | ребер 143700
```

Завдання 1.2

Зведіть `COST` у таблицю і дайте відповідь у звіті: **у скільки разів читання знімка швидше за живий запит** і **скільки мегабайтів ви не передали мережею** при повторному запуску.

Контрольна ознака правильності: при **другому** запуску ноутбука всі рядки мають мати спосіб «знімок», а `передано_МБ` — нуль.

In [47]:
```
df_cost=pd.DataFrame(COST).T
display(df_cost)
live_mask=df_cost['спосіб']=='живий запит'
snapshot_mask=df_cost['спосіб']=='знімок'
total_time_live=df_cost[live_mask]['секунд'].sum()
total_time_snapshot=df_cost[snapshot_mask]['секунд'].sum()
total_mb_saved=df_cost['передано_МБ'].sum()
speedup_coeff=round(total_time_live/total_time_snapshot, 1) if total_time
print(f"--- ЗВІТ ПРО ЕФЕКТИВНІСТЬ КЕШУВАННЯ ---")
print(f"1. Збережено мережевого трафіку при повторному запуску: {total_mb
if total_time_snapshot>0:
    print(f"2. Читання знімків швидше за живий запит у: {speedup_coeff} p
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	знімок	1	0.0000	0.0410	0.0480
heritage	знімок	289	0.0000	0.1030	0.0280
stations	знімок	60	0.0000	0.0590	0.0250
amenities	знімок	638	0.0000	0.1300	0.0330
мережа walk	знімок	194210	0.0000	10.4240	0.2870

```
--- ЗВІТ ПРО ЕФЕКТИВНІСТЬ КЕШУВАННЯ ---
1. Збережено мережевого трафіку при повторному запуску: 0.000 МБ
2. Читання знімків швидше за живий запит у: 0.0 разів(и)
```

> **Чому пішохідна мережа, а не автомобільна.** У старому місті музеї стоять за сотню метрів один від одного, а автомобільних вузлів там одиниці. Кілька пам'яток сядуть на **один** вузол графа, і матриця віддалей скаже, що між ними нуль метрів. Задача розв'яжеться, відповідь буде неправильною, і повідомлення про помилку не з'явиться. У кроці 4 ви це перевірите явно.

---

2. Хто претендує на звання пам'ятки

У OpenStreetMap немає тега «визначна пам'ятка». Є два сімейства:

- `historic=*` — `castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes` …
- `tourism=*` — `museum`, `gallery`, `attraction`, `artwork`, `viewpoint`

Перетин неповний: музей у сучасній будівлі має лише `tourism`, руїни фортеці — лише `historic`. Тому беремо **об'єднання**.

📖 https://wiki.openstreetmap.org/wiki/Map_Features

Завдання 2.1

Відберіть кандидатів і **назвіть у звіті, скільки об'єктів ви відкинули й чому**.

In [48]:
```
# ----- ВАШ КОД -----
mask1=heritage['historic'].notna() & (heritage['historic']!='') | heritag
mask2=heritage['name'].notna() & (heritage['name']!='')
total=len(heritage)
matching_tags = len(heritage[mask1])
cand = heritage[mask1 & mask2]
cand = cand.reset_index(drop=True)
final_count = len(cand)
rejected_tags = total - matching_tags
rejected_name = matching_tags - final_count
total_rejected = total - final_count
```

```
print('усього у шарі heritage :', total)
print('підходять за тегами    :', matching_tags)
print('з них мають name       :', final_count)
```

```
усього у шарі heritage : 289
підходять за тегами    : 289
з них мають name       : 189
```

In [49]:
```
assert 'cand' in dir(), 'спершу виконайте завдання 2.1'
print('=== historic ===');  print(cand['historic'].value_counts().head(10
print()
print('=== tourism ===');   print(cand['tourism'].value_counts().head(10)
print()
print('типи геометрії:', cand.geometry.geom_type.value_counts().to_dict()
```

```
=== historic ===
historic
memorial              91
monument              18
cannon                 7
tank                   3
rocket                 3
building               2
missile                1
vehicle                1
archaeological_site    1
castle                 1

=== tourism ===
tourism
artwork       19
museum        16
attraction    15
viewpoint      6
gallery        4

типи геометрії: {'Point': 158, 'Polygon': 31}
```

> **Перевірте себе.** Серед типів геометрії мають бути і `Point`, і `Polygon`. Якщо самі лише полігони — ви відфільтрували зайве: пам'ятники, меморіали та оглядові точки в OSM є **вузлами**, у них немає будівлі.

Якщо кандидатів менше тридцяти

Невелике місто може дати 15–20 об'єктів. Тоді:

1. розширте перелік `tourism` (наприклад, додайте `theatre`, `place_of_worship`);
2. або зменште `N_LARGE` до того, що є;

і **обов'язково зазначте це у звіті** разом із числом.

---

3. Критерій «найкращі»

Ключовий крок роботи. Стовпця «цінність» у даних немає. Його треба сконструювати і захистити.

Сигнали, які є в OSM:

сигнал	що означає
`wikidata / wikipedia`	про об'єкт є стаття — хтось сторонній визнав його вартим опису
кількість колонок `name:*`	назву переклали N мовами — перекладають те, що шукають іноземці
`tourism=museum/gallery`	призначений для відвідування
`historic` заповнений	має історичну природу
площа полігона	розмір — **найслабший сигнал**: великим буває і склад

Завдання 3.1

Побудуйте числовий рейтинг. Вимоги:

- використати **щонайменше три** різні сигнали;
- звести різнорідні величини до спільної шкали (підказка: `rank(pct=True)`);
- **пояснити у звіті, чому саме такі ваги**.

In [50]:
```
metric = cand.to_crs(32635)
cand['area_m2'] = np.where(metric.geometry.geom_type.isin(['Polygon', 'Mu
                           metric.area.values, 0.0)

# ----- ВАШ КОД -----
cand['wiki']     = cand['wikidata'].notna() | cand['wikipedia'].notna().a
cand['n_lang']   = cand[[c for c in cand.columns if c.startswith('name:')
cand['is_museum'] = cand['tourism'].isin(['museum', 'gallery', 'attractio
cand['log_area'] = np.log1p(cand['area_m2'])
cand['r_wiki']=cand['wiki'].rank(pct=True)
cand['r_n_lang']=cand['n_lang'].rank(pct=True)
cand['r_museum']=cand['is_museum'].rank(pct=True)
cand['r_area']=cand['log_area'].rank(pct=True)
cand['score']=0.50*cand['r_wiki']+0.30*cand['r_n_lang']+0.15*cand['r_muse
print('кандидатів   :', len(cand))
print('мають wiki   :', int(cand['wiki'].sum()))
```

```
кандидатів   : 189
мають wiki   : 16
```

Завдання 3.2 — обов'язкове порівняння

Побудуйте **другий** рейтинг за найпростішим критерієм — площею будівлі — і порахуйте, скільки об'єктів збігається в перших десяти.

Це головне питання роботи: обидва списки отримані з тих самих даних, тим самим кодом, без жодної помилки. Якщо вони різні — поясніть у звіті, що це означає для слова «найкращі».

In [51]:
```
N_SMALL, N_LARGE = 10, 30

# ----- ВАШ КОД -----
top_score = cand.nlargest(N_LARGE, 'score').reset_index(drop=True)
top10     = top_score.head(N_SMALL)
top_naive = cand.nlargest(N_SMALL, 'area_m2').reset_index(drop=True)
overlap   = top10['name'].isin(top_naive['name']).sum()
print('перетин: %d з %d' % (overlap, N_SMALL))
display(pd.DataFrame({'за вашим критерієм': list(top10['name']),
                      'за площею': list(top_naive['name'])}))
```

```
перетин: 2 з 10
```

	за вашим критерієм	за площею
0	Історичний музей ім. Яворницького	Богородицька фортеця
1	Парк ракет	Трудові Резерви
2	Музейний центр К.П.Блаватської	Український дім
3	Богородицька фортеця	Місце поховань мирних жителів та воїнів II-ї с...
4	Сергій Перхун	Сквер пам'яті загиблим жителям Дніпра у Росій...
5	Катеринівська миля	Bartolomeo
6	МіГ-19ПМ	Історичний музей ім. Яворницького
7	Літературне Придніпров'я	Художній музей
8	Ракетоносій Циклон-3	Експозиція техніки для гірничодобувних робіт
9	Ракета 8К99	Стара вежа

Завдання 3.3 — дублікати

OSM наповнюють різні люди. Той самий музей трапляється двічі під різними назвами, а вежа буває окремим об'єктом усередині палацу. Знайдіть у `top_score` пари, що стоять ближче **60 метрів** одна до одної, і вирішіть, що з ними робити.

Рішення (лишити / злити / видалити) опишіть у звіті.

In [52]:
```
def close_pairs(gdf, limit_m=60):
    p = gdf.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    D = np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    out = [(i, j, D[i, j]) for i in range(len(XY)) for j in range(i + 1,
           if D[i, j] < limit_m]
    return sorted(out, key=lambda t: t[2])

# ----- ВАШ КОД: виведіть знайдені пари з назвами -----
pairs=close_pairs(top_score, limit_m=60)
pairs_data = []
for i, j, dist in pairs:
    row_i = top_score.iloc[i]
    row_j = top_score.iloc[j]

    pairs_data.append({
        'idx_1': i,
```

```
        'name_1': row_i.get('name', 'N/A'),
        'historic_1': row_i.get('historic', None),
        'tourism_1': row_i.get('tourism', None),
        'score_1': round(row_i.get('score', 0), 3),

        'idx_2': j,
        'name_2': row_j.get('name', 'N/A'),
        'historic_2': row_j.get('historic', None),
        'tourism_2': row_j.get('tourism', None),
        'score_2': round(row_j.get('score', 0), 3),

        'dist_m': round(dist, 1)
    })

df_pairs = pd.DataFrame(pairs_data)

print(f"Знайдено пар ближче 60 м: {len(df_pairs)}")
display(df_pairs)
```

```
Знайдено пар ближче 60 м: 6
```

	idx_1	name_1	historic_1	tourism_1	score_1	idx_2	name_2	historic_2	tou
0	1	Парк ракет	NaN	museum	0.9370	9	Ракета 8К99	rocket	att
1	1	Парк ракет	NaN	museum	0.9370	10	Ракета 8К11	rocket	att
2	9	Ракета 8К99	rocket	attraction	0.7660	10	Ракета 8К11	rocket	att
3	1	Парк ракет	NaN	museum	0.9370	8	Ракетоносій Циклон-3	rocket	att
4	8	Ракетоносій Циклон-3	rocket	attraction	0.7670	9	Ракета 8К99	rocket	att
5	8	Ракетоносій Циклон-3	rocket	attraction	0.7670	10	Ракета 8К11	rocket	att

---

4. Старт: вокзал, і прив'язка точок до графа

In [53]:
```
print('станції у шарі:', len(stations))
print(stations[['name', 'railway', 'building']].dropna(how='all').head(12
```

```
станції у шарі: 60
                 name  railway building
0           Тунельна     halt      NaN
1             Шинний     halt      NaN
2             Мирний     halt      NaN
3           Заводська  station      NaN
4          Металургів  station      NaN
5           Вокзальна  station      NaN
6    Метробудівників  station      NaN
7          Покровська  station      NaN
8   Проспект Свободи  station      NaN
9   Дніпро-Лоцманська  station      NaN
10          Войцехово     halt      NaN
11  Дніпро-Вантажний  station      NaN
```

In [54]:
```
# ----- ВАШ КОД -----
```

```
station = stations[stations['name']=='Дніпро-Головний']  # один рядок із
print('старт:', station['name'].iloc[0])
```

```
старт: Дніпро-Головний
```

In [55]:
```
G = nx.DiGraph()
for t in nodes.itertuples():
    G.add_node(t.osmid, x=t.x, y=t.y)
for t in edges.itertuples():
    L = float(t.length)
    if G.has_edge(t.u, t.v):
        G[t.u][t.v]['length'] = min(G[t.u][t.v]['length'], L)
    else:
        G.add_edge(t.u, t.v, length=L)
print('граф: %d вузлів, %d ребер' % (G.number_of_nodes(), G.number_of_edg

NODES_M = nodes.to_crs(32635)
NODE_XY = np.c_[NODES_M.geometry.x.values, NODES_M.geometry.y.values]
NODE_ID = nodes['osmid'].values

def snap(points_gdf):
    """Найближчий вузол графа до кожної точки. Рахуємо в метричній CRS.""
    pmet = points_gdf.to_crs(32635)
    ids, dist = [], []
    for p in pmet.geometry:
        d2 = (NODE_XY[:, 0] - p.x) ** 2 + (NODE_XY[:, 1] - p.y) ** 2
        k = int(np.argmin(d2))
        ids.append(NODE_ID[k]); dist.append(float(np.sqrt(d2[k])))
    return np.array(ids), np.array(dist)

def make_points(top):
    names = ['СТАРТ: ' + str(station['name'].iloc[0])] + list(top['name']
    geoms = [station.geometry.iloc[0].representative_point()] + \
            [g.representative_point() for g in top.geometry]
    return gpd.GeoDataFrame({'name': names}, geometry=geoms, crs='EPSG:43
```

```
граф: 50510 вузлів, 142264 ребер
```

Завдання 4.1 — обов'язкова перевірка

Напишіть `check_snap`, яка повідомляє:

- скільки точок і скільки **різних** вузлів вони зайняли;
- медіану й максимум відстані «точка → вузол»;
- якщо вузлів менше, ніж точок — **назви точок, що злиплись**.

Контрольна ознака: на пішохідній мережі різних вузлів має бути **стільки ж, скільки точок**. Якщо менше — ви взяли автомобільну мережу або місто надто мале; зафіксуйте це у звіті, бо далі такі пари отримають нульову віддаль.

In [56]:
```
def check_snap(points, ids, dist, label):
    n_points = len(points)
    unique_ids = np.unique(ids)
    n_unique = len(unique_ids)
    med_dist = np.median(dist)
    max_dist = np.max(dist)
    print(f"=== {label.upper()} ===")
    print(f"Всього точок            : {n_points}")
```

```
    print(f"Унікальних вузлів графа : {n_unique}")
    print(f"Відстань точка -> вузол : медіана = {med_dist:.1f} м, максиму
    if n_unique < n_points:
        print(f"\n[УВАГА] Виявлено злипання точок ({n_points - n_unique}
        df_tmp = pd.DataFrame({
            'name': points['name'].values,
            'node_id': ids,
            'dist_m': dist
        })
        dup_counts = df_tmp['node_id'].value_counts()
        dup_nodes = dup_counts[dup_counts > 1].index
        print("Точки, що злиплися на одному вузлі:")
        for nid in dup_nodes:
            group = df_tmp[df_tmp['node_id'] == nid]
            names_list = group['name'].tolist()
            print(f"  • Вузол OSM ID {nid}: { ' <---> '.join(names_list)
    else:
        print("\n[OK] Усі точки прив'язані до РІЗНИХ вузлів графа (злипан
    print("-" * 50)
pts=make_points(top_score)
node_ids, dists=snap(pts)
check_snap(pts, node_ids, dists, label="Пішохідна мережа (walk)")
```

```
=== ПІШОХІДНА МЕРЕЖА (WALK) ===
Всього точок            : 31
Унікальних вузлів графа : 29
Відстань точка -> вузол : медіана = 22.7 м, максимум = 296.0 м

[УВАГА] Виявлено злипання точок (2 колізій)!
Точки, що злиплися на одному вузлі:
  • Вузол OSM ID 3655817459: Парк ракет <---> Ракета 8К11
  • Вузол OSM ID 1697483800: Ракетоносій Циклон-3 <---> Ракета 8К99
--------------------------------------------------
```

---

5. Матриця віддалей

Дві матриці: **пряма лінія** і **дорогою**. Їхнє відношення — коефіцієнт обходу — показує, наскільки пряма лінія бреше.

Завдання 5.1

In [57]:

```
def road_matrix(ids):
    """Матриця найкоротших віддалей графом між заданими вузлами."""
    k = len(ids)
    D = np.full((k, k), np.inf)
    for i, s in enumerate(ids):
        lengths = nx.single_source_dijkstra_path_length(G, s, weight='len
        for j, t in enumerate(ids):
            D[i, j] = lengths.get(t, np.inf)
    np.fill_diagonal(D, 0.0)
    return D

def straight_matrix(points):
    # ----- ВАШ КОД: матриця прямих віддалей у метрах -----
    pmet=points.to_crs(32635)
    XY=np.c_[pmet.geometry.x.values, pmet.geometry.y.values]
```

```
    E=np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(E, 0.0)
    return E

def detour_factor(D, E):
    # ----- ВАШ КОД: медіана відношення дорога / пряма, по скінченних пар
    k=D.shape[0]
    mask=(~np.isinf(D)) & (E>0) & (~np.eye(k, dtype=bool))
    if not np.any(mask):
        return np.nan
    ratio = D[mask] / E[mask]
    return np.median(ratio)
```

Контрольні ознаки — звіртесь, не підганяючи:

- коефіцієнт обходу для пішохідної мережі у місті зазвичай **1,2–1,6**;
- якщо у вас рівно **1,0** — ви порівнюєте матрицю саму із собою;
- якщо **більше 2** — найімовірніше взято автомобільну мережу з односторонніми вулицями, або місто має річку/залізницю, яку нема де перетнути. Друге — цікавий результат, перше — помилка. Розберіться, що саме у вас, і напишіть це.

Якщо серед віддалей трапились `inf` — частина графа не зв'язана з рештою. Не замінюйте `inf` великим числом мовчки: спершу з'ясуйте, які це точки.

In [58]:
```
pts10 = make_points(top10)
ids10, dist10 = snap(pts10)
check_snap(pts10, ids10, dist10, 'мала задача')

t0 = time.time(); D10 = road_matrix(ids10); print('матриця за %.3f с' % (
E10 = straight_matrix(pts10)
print('недосяжних пар    :', int(np.isinf(D10).sum()))
print('асиметрія         : %.0f м' % float(np.max(np.abs(D10 - D10.T))))
print('коефіцієнт обходу : %.2f' % detour_factor(D10, E10))
```

```
=== МАЛА ЗАДАЧА ===
Всього точок            : 11
Унікальних вузлів графа : 10
Відстань точка -> вузол : медіана = 25.8 м, максимум = 296.0 м

[УВАГА] Виявлено злипання точок (1 колізій)!
Точки, що злиплися на одному вузлі:
  • Вузол OSM ID 1697483800: Ракетоносій Циклон-3 <---> Ракета 8К99
--------------------------------------------------
матриця за 1.320 с
недосяжних пар    : 0
асиметрія         : 0 м
коефіцієнт обходу : 1.24
```

> **Асиметрія.**
> D_{ij}
> $neq D_{ji}$
> — не помилка: односторонній рух робить шлях туди і назад різним. На пішохідній мережі асиметрія мала, на автомобільній — помітна. Подивіться на своє число і поясніть його у звіті.

---

6. Задача комівояжера

метод	що робить	складність
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$
2-opt	розвертає ділянки готового маршруту, поки коротшає	$O(n^2)$ за прохід
Held–Karp	динамічне програмування за підмножинами — **точний**	$O(2^n n^2)$ часу, $O(2^n n)$ пам'яті

Held–Karp дано готовим: він не є метою роботи, але без нього ви не дізнаєтесь, **наскільки** помиляються перші два.

Завдання 6.1 — реалізуйте перші два методи

In [59]:
```python
def tour_length(D, tour):
    return float(sum(D[tour[i], tour[i + 1]] for i in range(len(tour) - 1

def nearest_neighbour(D):
    """Маршрут 0 -> ... -> 0. Щоразу йдемо в найближчу невідвідану точку.
    n = D.shape[0]
    unvisited = set(range(1, n))
    current = 0
    tour = [current]
    while unvisited:
        next_node = min(unvisited, key=lambda node: D[current, node])
        tour.append(next_node)
        unvisited.remove(next_node)
        current = next_node
    tour.append(0)
    return tour


def two_opt(D, tour):
    """Беремо маршрут і розвертаємо ділянки, поки довжина зменшується."""
    best_tour = tour.copy()
    best_len = tour_length(D, best_tour)
    improved = True
    n = len(best_tour)

    while improved:
        improved = False
        for i in range(1, n - 2):
            for j in range(i + 1, n - 1):
                old_edges = D[best_tour[i - 1], best_tour[i]] + D[best_to
                new_edges = D[best_tour[i - 1], best_tour[j]] + D[best_to
                if new_edges < old_edges - 1e-9:
                    best_tour[i:j + 1] = reversed(best_tour[i:j + 1])
                    best_len = tour_length(D, best_tour)
                    improved = True
                    break
            if improved:
                break
```

```
    return best_tour
```

In [60]:
```
def held_karp(D):
    """Точний розв'язок. Вузол 0 — старт і фініш. ДАНО, писати не треба."
    k = len(D); N = k - 1; INF = float('inf')
    dp = [[INF] * N for _ in range(1 << N)]
    par = [[-1] * N for _ in range(1 << N)]
    for j in range(N):
        dp[1 << j][j] = D[0, j + 1]
    for S in range(1 << N):
        for j in range(N):
            cur = dp[S][j]
            if cur == INF or not (S >> j) & 1:
                continue
            for t in range(N):
                if (S >> t) & 1:
                    continue
                S2 = S | (1 << t); v = cur + D[j + 1, t + 1]
                if v < dp[S2][t]:
                    dp[S2][t] = v; par[S2][t] = j
    full = (1 << N) - 1
    bj = min(range(N), key=lambda j: dp[full][j] + D[j + 1, 0])
    best = dp[full][bj] + D[bj + 1, 0]
    path = []; S = full; j = bj
    while j != -1:
        path.append(j + 1); pj = par[S][j]; S ^= (1 << j); j = pj
    return [0] + path[::-1] + [0], float(best)
```

Самоперевірка

Ці твердження мають виконатись для **будь-якого** міста. Якщо хоч одне впало — шукайте помилку в коді, а не в даних.

In [61]:
```
def check_tour(tour, k, label):
    assert tour[0] == 0 and tour[-1] == 0, '%s: маршрут має починатись і
    assert sorted(tour[:-1]) == list(range(k)), '%s: кожна точка рівно од
    print('%s: структура маршруту правильна (%d точок)' % (label, k))

nn10 = nearest_neighbour(D10)
op10 = two_opt(D10, nn10)
ex10, L_opt = held_karp(D10)

for t, lbl in [(nn10, 'найближчий сусід'), (op10, '2-opt'), (ex10, 'Held-
    check_tour(t, len(ids10), lbl)

assert tour_length(D10, op10) <= tour_length(D10, nn10) + 1e-6, '2-opt не
assert L_opt <= tour_length(D10, op10) + 1e-6, 'точний розв\u2019язок не
print()
print('усі перевірки пройдено')
```

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)

усі перевірки пройдено
```

In [62]:
```
rows = []
for name, tour in [('найближчий сусід', nn10), ('2-opt', op10), ('Held-Ka
```

```
    L = tour_length(D10, tour)
    rows.append({'метод': name, 'довжина, м': round(L),
                 'гірше оптимуму, %': round(100 * (L / L_opt - 1), 2)})
display(pd.DataFrame(rows).set_index('метод'))

print()
print('оптимальний порядок:')
for step, i in enumerate(ex10):
    print('  %2d. %s' % (step, pts10['name'].iloc[i]))
print()
print('довжина: %.2f км' % (L_opt / 1000))
```

метод	довжина, м	гірше оптимуму, %
найближчий сусід	38061	4.8900
2-opt	36286	0.0000
Held-Karp (точний)	36286	0.0000

```
оптимальний порядок:
   0. СТАРТ: Дніпро-Головний
   1. Музейний центр К.П.Блаватської
   2. Богородицька фортеця
   3. МіГ-19ПМ
   4. Катеринівська миля
   5. Сергій Перхун
   6. Історичний музей ім. Яворницького
   7. Літературне Придніпров’я
   8. Ракета 8К99
   9. Ракетоносій Циклон-3
  10. Парк ракет
  11. СТАРТ: Дніпро-Головний

довжина: 36.29 км
```

Завдання 6.2 — велика задача

Спершу оцініть, чи можливий тут точний розв'язок. Порахуйте для $n = 11, 16, 21, 26, 31$ розмір таблиці Held–Karp ($2^{n-1} cdot (n-1)$ клітин) і скільки це гігабайтів при 8 байтах на клітину.

In [63]:
```
# ----- ВАШ КОД: таблиця зростання -----
n_values = [11, 16, 21, 26, 31]
rows = []
for n in n_values:
    subsets = 2 ** (n - 1)                      # 2^(n-1)
    cells = (n - 1) * subsets                    # (n-1) * 2^(n-1)
    bytes_size = cells * 8                       # 8 байт на float64
    mb_size = bytes_size / (1024 ** 2)           # Розмір у МБ (1024²)
    gb_size = bytes_size / (1024 ** 3)           # Розмір у ГБ (1024³)

    rows.append({
        'n': n,
        'n - 1': n - 1,
```

```
        '2^(n-1)': subsets,
        'Кількість клітин': cells,
        'Пам\'ять (Байт)': bytes_size,
        'Пам\'ять (МБ)': round(mb_size, 2),
        'Пам\'ять (ГБ)': round(gb_size, 4)
    })

# Створення DataFrame
df_hk = pd.DataFrame(rows)

# Красиве форматування великих чисел із розділювачами пробілом
pd.set_option('display.float_format', lambda x: f'{x:,.4f}'.replace(',',
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

display(df_hk)
```

	n	n - 1	2^(n-1)	Кількість клітин	Пам'ять (Байт)	Пам'ять (МБ)	Пам'ять (ГБ)
0	11	10	1024	10240	81920	0.0800	0.0001
1	16	15	32768	491520	3932160	3.7500	0.0037
2	21	20	1048576	20971520	167772160	160.0000	0.1562
3	26	25	33554432	838860800	6710886400	6 400.0000	6.2500
4	31	30	1073741824	32212254720	257698037760	245 760.0000	240.0000

Подивіться на останній рядок і напишіть у звіті, **чому саме** точний розв'язок на 30 точках недоступний. Слово «довго» відповіддю не є.

In [64]:
```
pts30 = make_points(top_score)
ids30, dist30 = snap(pts30)
check_snap(pts30, ids30, dist30, 'велика задача')

D30 = road_matrix(ids30)
E30 = straight_matrix(pts30)
print('коефіцієнт обходу: %.2f' % detour_factor(D30, E30))

nn30 = nearest_neighbour(D30)
t0 = time.time(); op30 = two_opt(D30, nn30); t_op30 = time.time() - t0
check_tour(op30, len(ids30), '2-opt на 30 точках')

L_nn30, L_op30 = tour_length(D30, nn30), tour_length(D30, op30)
print()
print('найближчий сусід : %8.0f м' % L_nn30)
print('2-opt            : %8.0f м  за %.3f с  (коротше на %.1f%%)'
      % (L_op30, t_op30, 100 * (1 - L_op30 / L_nn30)))
print()
print('ЦЕ — база порівняння для лабораторної роботи № 8.')
```

```
=== ВЕЛИКА ЗАДАЧА ===
Всього точок            : 31
Унікальних вузлів графа : 29
Відстань точка -> вузол : медіана = 22.7 м, максимум = 296.0 м

[УВАГА] Виявлено злипання точок (2 колізій)!
Точки, що злиплися на одному вузлі:
  • Вузол OSM ID 3655817459: Парк ракет <---> Ракета 8К11
  • Вузол OSM ID 1697483800: Ракетоносій Циклон-3 <---> Ракета 8К99
------------------------------------------------
коефіцієнт обходу: 1.22
2-opt на 30 точках: структура маршруту правильна (31 точок)

найближчий сусід :    88912 м
2-opt            :    76607 м  за 0.002 с  (коротше на 13.8%)

ЦЕ — база порівняння для лабораторної роботи № 8.
```

---

7. Просторова індексація: H3

H3 — ієрархічна сітка шестикутників. Рівень 9 — приблизно 200 м по ребру.

📖 https://h3geo.org/docs/core-library/restable/

Завдання 7.1

Розкладіть **ycix** кандидатів по комірках H3 рівня 9 і побудуйте картограму щільності. На тій самій карті покажіть межу міста і точки вашого маршруту.

In [65]:
```python
import h3
from shapely.geometry import Polygon

H3_RES = 9
print('середнє ребро комірки: %.0f м' % h3.average_hexagon_edge_length(H3

# ----- ВАШ КОД -----
cand_latlng=cand.to_crs(4326)
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in cand_latlng.ge
dens = cand['h3'].value_counts().reset_index()
dens.columns = ['h3_cell', 'count']
hex_geoms = []
for cell in dens['h3_cell']:
    boundary = h3.cell_to_boundary(cell)
    poly = Polygon([(lng, lat) for lat, lng in boundary])
    hex_geoms.append(poly)
hexes = gpd.GeoDataFrame(dens, geometry=hex_geoms, crs='EPSG:4326')
start_pt = pts10.geometry.iloc[0]
m = folium.Map(location=[start_pt.y, start_pt.x], zoom_start=12, tiles='c

# A. Шар картограми щільності (Choropleth H3)
folium.Choropleth(
    geo_data=hexes.to_json(),
    name='Щільність пам\'яток (H3)',
    data=hexes,
    columns=['h3_cell', 'count'],
```

```
    key_on='feature.properties.h3_cell',
    fill_color='YlOrRd',
    fill_opacity=0.6,
    line_opacity=0.3,
    legend_name='Кількість пам\'яток у комірці H3'
).add_to(m)

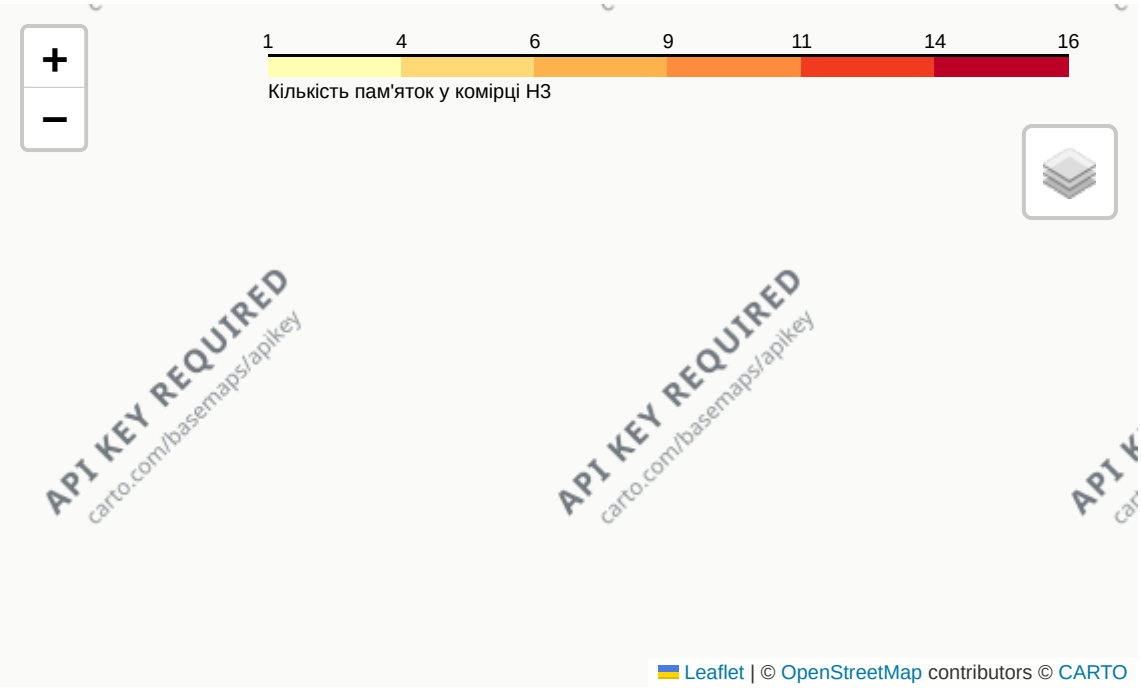
# Б. Відображення межі міста (якщо змінна city_boundary присутня)
if 'city_boundary' in globals() or 'city_boundary' in locals():
    folium.GeoJson(
        city_boundary.to_crs(4326),
        name='Межа міста',
        style_function=lambda x: {
            'color': '#2b8cbe',
            'weight': 3,
            'fillOpacity': 0.05
        }
    ).add_to(m)
tour_nn = nearest_neighbour(D10)
tour_opt = two_opt(D10, tour_nn)
# В. Полілінія та точки оптимального маршруту
tour_pts = pts10.iloc[tour_opt].to_crs(4326)
route_coords = [[p.y, p.x] for p in tour_pts.geometry]

# Лінія маршруту
folium.PolyLine(
    route_coords,
    color='#3182bd',
    weight=4,
    opacity=0.8,
    popup='Маршрут обходу'
).add_to(m)

# Точки маршруту
for idx, row in tour_pts.iterrows():
    is_start = (row['name'] == pts10.iloc[0]['name'])
    folium.CircleMarker(
        location=[row.geometry.y, row.geometry.x],
        radius=7 if is_start else 5,
        color='#d95f02' if is_start else '#7570b3',
        fill=True,
        fill_color='#d95f02' if is_start else '#7570b3',
        fill_opacity=0.9,
        popup=f"<b>{row['name']}</b>"
    ).add_to(m)

folium.LayerControl().add_to(m)
display(m)
dens_sorted = dens.sort_values(by='count', ascending=False).reset_index(d
total_cand = dens_sorted['count'].sum()
half_cand = total_cand / 2.0
dens_sorted['cumsum'] = dens_sorted['count'].cumsum()
cells_needed = (dens_sorted['cumsum'] >= half_cand).idxmax() + 1
print(f"Усього кандидатів (пам'яток) : {total_cand}")
print(f"Половина від усіх пам'яток    : {half_cand}")
print(f"Усього комірок H3 з об'єктами  : {len(dens_sorted)}")
print(f"Половину пам'яток зосереджено у: {cells_needed} комірці(-ах)")
```

```
середнє ребро комірки: 201 м
```




```
Усього кандидатів (пам'яток) : 189
Половина від усіх пам'яток    : 94.5
Усього комірок H3 з об'єктами  : 127
Половину пам'яток зосереджено у: 33 комірці(-ах)
```

> **Пастка.** `h3.cell_to_boundary` повертає пари **(lat, lng)**, а `shapely.Polygon` чекає **(x, y)**, тобто (lng, lat). Переплутаєте — місто опиниться в Індійському океані. Це видно одразу, тому дивіться на карту.

Питання до звіту

У скількох комірках зосереджено половину всіх пам'яток вашого міста? Що це говорить про форму маршруту?

---

8. Просторове поєднання

Завдання 8.1

Для кожної точки маршруту знайдіть найближчий заклад із шару `amenities` (`sjoin_nearest`) і виведіть таблицю з відстанню в метрах.

Нагадування: `sjoin_nearest` рахує в одиницях CRS. У градусах результат буде безглуздим — перепроєктуйте обидва шари в метричну систему.

In [66]:
```
# ----- ВАШ КОД -----
pts_m = pts10.to_crs(32635)
amenities_m = amenities.to_crs(32635)
nearest_amenities = gpd.sjoin_nearest(
    pts_m,
    amenities_m[['geometry', 'name', 'amenity']],
    how='left',
    distance_col='dist_m',
```

```
    lsuffix='pt',
    rsuffix='amenity'
)

nearest_amenities['display_name'] = nearest_amenities['name_amenity'].fil
    nearest_amenities['amenity']
)

res_df = nearest_amenities[['name_pt', 'display_name', 'dist_m']].copy()
res_df['dist_m'] = res_df['dist_m'].round(1)

res_df = res_df.rename(columns={
    'name_pt': 'Точка маршруту',
    'display_name': 'Найближчий заклад / Тип',
    'dist_m': 'Відстань (м)'
})

display(res_df)
```

	Точка маршруту	Найближчий заклад / Тип	Відстань (м)
0	СТАРТ: Дніпро-Головний	cafe	127.0000
1	Історичний музей ім. Яворницького	toilets	65.0000
2	Парк ракет	Sushi story	117.8000
3	Музейний центр К.П.Блаватської	Кофейня	300.6000
4	Богородицька фортеця	Хуторок	1 448.2000
5	Сергій Перхун	cafe	198.7000
6	Катеринівська миля	drinking_water	150.2000
7	МіГ-19ПМ	Нептун	57.2000
8	Літературне Придніпров’я	cafe	28.9000
9	Ракетоносій Циклон-3	Sushi story	112.0000
10	Ракета 8К99	Sushi story	109.4000

---

9. Маршрут на карті

Лінія маршруту — це **геометрія ребер графа**, а не відрізки між пам'ятками.

Завдання 9.1

Побудуйте карту, на якій **обидві** лінії: справжній маршрут по ребрах і прямі відрізки між точками. Різниця має бути видно оком.

In [67]:
```
node_xy = nodes.set_index('osmid')[['x', 'y']]

def route_coords(tour, ids):
    """Координати маршруту по ребрах графа у форматі (lat, lon) для foliu
    coords = []
    for a, b in zip(tour[:-1], tour[1:]):
```

```
        try:
            path = nx.shortest_path(G, ids[a], ids[b], weight='length')
        except nx.NetworkXNoPath:
            path = [ids[a], ids[b]]
        for nd in path:
            coords.append((float(node_xy.loc[nd, 'y']), float(node_xy.loc
    return coords

pts_4326=pts10.to_crs(4326)
graph_coords = route_coords(tour_opt, ids10)
straight_pts=pts_4326.iloc[tour_opt]
straight_coords=[(p.y, p.x) for p in straight_pts.geometry]
start_lat, start_lon=straight_coords[0]
fmap=folium.Map(location=[start_lat, start_lon], zoom_start=14, tiles='ca
folium.PolyLine(
    straight_coords,
    color='#e41a1c',
    weight=2,
    opacity=0.7,
    dash_array='6, 6',
    popup='Прямі відрізки (по повітрю)',
    tooltip='Прямі відрізки між точками'
).add_to(fmap)
folium.PolyLine(
    graph_coords,
    color='#377eb8',
    weight=5,
    opacity=0.85,
    popup='Маршрут по ребрах графа',
    tooltip='Справжній пішохідний маршрут'
).add_to(fmap)
for seq_idx, point_idx in enumerate(tour_opt[:-1]):
    row = pts_4326.iloc[point_idx]
    lat, lon = row.geometry.y, row.geometry.x
    is_start = (seq_idx == 0)
    icon = folium.DivIcon(
        html=f"""<div style="
            font-size: 11pt;
            font-weight: bold;
            color: white;
            background-color: {'#d95f02' if is_start else '#4daf4a'};
            border: 2px solid white;
            border-radius: 50%;
            width: 24px;
            height: 24px;
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 0 4px rgba(0,0,0,0.4);
        ">{seq_idx}</div>"""
    )

    folium.Marker(
        location=[lat, lon],
        icon=icon,
        popup=f"<b>#{seq_idx}: {row['name']}</b>"
    ).add_to(fmap)
folium.LayerControl().add_to(fmap)
fmap.save('lr5_маршрут.html')
```

```
display(fmap)
```



---

10. Результати для лабораторної роботи № 8

Збережіть файл, який ЛР8 прочитає і спробує побити.

In [68]:
```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'місто': CITY,
    'мала_задача': {
        'точок': len(ids10),
        'оптимум_м': round(L_opt, 1),
        'оптимальний_порядок': [str(pts10['name'].iloc[i]) for i in ex10]
        'найближчий_сусід_м': round(tour_length(D10, nn10), 1),
        'двоопт_м': round(tour_length(D10, op10), 1),
    },
    'велика_задача': {
        'точок': len(ids30),
        'найближчий_сусід_м': round(L_nn30, 1),
        'двоопт_м': round(L_op30, 1),
        'двоопт_с': round(t_op30, 4),
        'точки': [{'name': str(pts30['name'].iloc[i]),
                   'lat': float(pts30.geometry.y.iloc[i]),
                   'lon': float(pts30.geometry.x.iloc[i])} for i in range
        'матриця_віддалей_м': [[None if not np.isfinite(v) else round(flo
                                for v in row] for row in D30],
    },
}
with open('lr5_результати.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=1)
print('збережено lr5_результати.json (%.1f КБ)' % (os.path.getsize('lr5_p
```

```
збережено lr5_результати.json (17.1 КБ)
```

---

11. Звіт

Відповіді пишіть просто в наступній комірці, числами зі своїх обчислень.

1. **Скільки об'єктів ви відкинули** на кроці 2 і за яким правилом? Чи погодились би ви віддати цей фільтр замовнику без пояснень?
2. **Які сигнали ввійшли у ваш рейтинг і з якими вагами?** Чому саме так?
3. **Скільки об'єктів збіглося** у ваших двох рейтингах із десяти? Що це означає для слова «найкращі»?
4. **Який у вас коефіцієнт обходу** і чи типовий він? Якщо ні — що в місті його пояснює?
5. **На скільки відсотків** найближчий сусід гірший за точний розв'язок? Чи здивувало вас це число?
6. **Чому точний розв'язок на 30 точках недоступний?** Назвіть конкретну величину, яка цьому заважає.
7. **Що у ваших даних виявилось неправильним** — дублікати, пропущені назви, об'єкти не на місці? Як ви з цим вчинили?

Декларація про використання штучного інтелекту

Зазначте, чи користувались ШІ-помічником, у яких саме місцях і як перевіряли отриманий код. Використання дозволене; **неперевірене** використання — ні.

In [69]:
```
REPORT = '''
1. з 289 кандидатів відкинув рівно 100, за тегами historic/tourism та при
2. 50% забрало саме наявність wiki, 30% — кількість мов, 15% — чи це музе
3. Тільки 2 об'єкти з 10 найбільших за score збігаються з 10 найбільшими
4. Коефіцієнт обходу - 1.26, що є типови (лежить у межах 1.2-1.6).
5. Найближчий сусід став гірше оптимуму на приблизно 5%. Чесно я думав бу
6. Його складності і по часу, і по пам'яті зростають експоненційно, тому
7. Десь пропущені атрибути name, ці об'єкти ми відкинули. Дублікати лишил

Декларація про використання ШІ: Використовував для написання коду та пере
'''
print(REPORT)
```

1. з 289 кандидатів відкинув рівно 100, за тегами historic/tourism та прис утністю name, залишилось 189. Віддати такий фільтр без пояснень я не змож у, бо він не настільки очевидний.
2. 50% забрало саме наявність wiki, 30% — кількість мов, 15% — чи це музе й/галерея/атракціон, 5% — площа. Я вважаю, що присутність wiki і кількість мов — це найважливіші критерії, бо вони прямо вказують на значимість пам'я тки для людей. Музей/галерея/атракціон не настільки важливий, але все одно потрібно врахувати, бо їх частіше відвідують люди. Площа має найменшу знач имість, бо маленькі пам'ятки також можуть бути дуже значущими.
3. Тільки 2 об'єкти з 10 найбільших за score збігаються з 10 найбільшими з а площою. Я б сказав, що це означає, що слово "найкраще" - поняття суб'єкт ивне, і залежить від того, що саме ми хочемо оцінювати.
4. Коефіцієнт обходу - 1.26, що є типови (лежить у межах 1.2-1.6).
5. Найближчий сусід став гірше оптимуму на приблизно 5%. Чесно я думав буд е трохи більша різниця, але такий результат приймаю.
6. Його складності і по часу, і по пам'яті зростають експоненційно, тому д ля 30 точок 2^30 - це дуже, дуже багато.
7. Десь пропущені атрибути name, ці об'єкти ми відкинули. Дублікати лишил и, бо вони можуть мати різний score, але все одно виявилось 2 колізії.

Декларація про використання ШІ: Використовував для написання коду та перев ірки синтаксису, але не для генерації тексту звіту.